

#PyTorch » Feed PyTorch » Comments Feed PyTorch » iCal Feed alternate

IFRAME: <https://www.googletagmanager.com/ns.html?id=GTM-T8XT4PS>

[Skip to main content](#)

Join us at PyTorch Conference North America • Oct 20-21 • San Jose, CA
Start Typing... (BUTTON) Search
[Close Search](#)

PyTorch Logo
[search](#)
[Menu](#)

- * [Learn](#)
 - + [Get Started](#)
 - + [Tutorials](#)
 - + [Learn the Basics](#)
 - + [PyTorch Recipes](#)
 - + [Intro to PyTorch »\200\223 YouTube Series](#)
 - + [Videos](#)
 - + [PyTorch Certification](#)
- * [Community](#)
 - + [Landscape](#)
 - + [Join the Ecosystem](#)
 - + [Community Hub](#)
 - + [Forums](#)
 - + [Developer Resources](#)
 - + [Events](#)
 - + [Working Groups](#)
 - + [Meeting Calendar](#)
 - + [Contributor Awards](#)
 - + [Ambassadors](#)
- * [Projects](#)
 - + [PyTorch](#)
 - o [Executorch](#)
 - + [vLLM](#)
 - + [DeepSpeed](#)
 - + [Ray](#)
 - + [Helion](#)
 - + [Safetensors](#)
 - + [Host Your Project](#)
- * [Docs](#)
 - + [PyTorch](#)
 - + [Domains](#)
- * [Blog & News](#)
 - + [Blog](#)
 - + [Announcements](#)
 - + [Case Studies](#)
 - + [Newsletter](#)
- * [About](#)
 - + [PyTorch Foundation](#)
 - + [Members](#)
 - + [Governing Board](#)
 - + [Technical Advisory Council](#)
 - + [Cloud Credit Program](#)
 - + [Staff](#)
 - + [Contact](#)
 - + [Brand Guidelines](#)
- * [JOIN](#)
- * [search](#)

[Blog](#)

Generating State-of-the-Art GEMMs with TorchInductor »\200\231s CudaDSL backend

By [Nikhil Patel](#), [Michael Lazos](#), [Driss Guessous](#), [Elias Ellison](#), [Meta](#) April 7, 2026 April 13th, 2026 No Comments

Introduction

TorchInductor currently supports three autotuning backends for matrix multiplications: Triton, CUTLASS (C++), and cuBLAS. This post describes the integration of CudaDSL as a fourth backend, the technical motivation for the work, and the performance results observed so far.

The kernel-writing DSL space has gained significant momentum, with Triton, Helion, Gluon, CuTile, and CudaDSL each occupying a different point in the abstraction-performance tradeoff. When evaluating whether to integrate a new backend into TorchInductor, we apply three criteria: (1) the integration does not impose a large maintenance burden on our team, or there is a long-term committed effort from the vendor; (2) it does not regress compile time or benchmarking time relative to existing backends; and (3) it delivers better performance on target workloads.

CudaDSL satisfies all three. NVIDIA is actively developing CudaDSL and provides optimized kernel templates, which limits the maintenance burden on TorchInductor. Compile times are at parity with our other backends, a significant improvement over the CUTLASS C++ path which requires full nvcc invocations.

Beyond these immediate benefits, CudaDSL represents a longer-term strategic investment. It is built on the same abstractions as CUTLASS C++, which has demonstrated strong performance on FP8 GEMMs and epilogue fusion, but it is written in Python, has faster compile times, and is less complex to maintain. As NVIDIA continues to invest in CudaDSL performance, CudaDSL is positioned to serve as an eventual replacement for the CUTLASS C++ integration on newer hardware generations, simplifying the TorchInductor codebase. The combination of aligned incentives, growing open-source adoption (Tri Dao »\200\231s Quack library, Jay Shah at Colfax International), and a

lower-level programming model that exposes the full thread and memory hierarchy makes CudaDSL a well-positioned backend for delivering optimal GEMM performance on current and future NVIDIA hardware.

Strategy: Why We Target GEMMs

Not all operations benefit equally from a new backend. For memory-bound operations like elementwise math, activations, and reductions, Triton already generates high-quality code. Its block-level programming model is well-suited to these workloads which only require vectorized memory accesses, and the performance gap between Triton and hand-written kernels is small. CudaDSL can express pointwise operations and reductions, but due to its low-level nature, automatically generating CudaDSL kernels from scratch is complex. In practice the two DSLs produce kernels that perform comparably on these workloads, so this extra complexity would not provide any benefit. Our own experiments are shown below which validate this theory. We ran a triton and cudaDSL softmax kernel on progressively larger input sizes both approach terminal bandwidth on GB200.

GEMMs are a different story. Matrix multiplications dominate the compute profile of transformer-based models: in a typical LLM forward pass, GEMMs in the attention projections, FFN layers, and output head account for the majority of GPU cycles. Achieving near-peak utilization on these operations requires precise control over the hardware features that each new GPU generation introduces like tile sizes tuned to the tensor core pipeline, explicit management of shared memory staging, warp-level scheduling, and on newer architectures like B200, thread block clusters and distributed shared memory. These are exactly the concerns that higher-level languages abstract away for ease of use. To simplify generating the low-level code, we avoid generating the kernel from scratch by starting with hand-optimized templates which expose the tunable parameters needed for adapting performance to different problem shapes.

The existing CUTLASS C++ backend addresses this by providing low-level control, but the C++ compilation overhead creates practical limitations: each kernel variant requires a full nvcc invocation, making it expensive to evaluate many candidates during autotuning and impractical to benchmark epilogue fusion decisions at scheduling time.

CudaDSL resolves this issue via a custom Python to MLIR compiler. The DSL itself is built on the same abstractions as CUTLASS C++ like the same tile algebra, the same memory hierarchy primitives, the same epilogue fusion model but compiles at speeds comparable to TorchInductor's other backends. This combination makes it

possible to apply the full autotuning and benchmark fusion pipeline that TorchInductor uses for other backends to GEMM kernels that have CUTLASS-level hardware control. The specific properties that enable this are:

Full thread and memory hierarchy exposure. CudaDSL provides primitives for synchronization, warp-level control, thread block clusters, and the complete thread/memory hierarchy. This enables use of architecture-specific features such as distributed shared memory on H100 and B200.

Compile time improvements. The CUTLASS C++ path requires a full nvcc invocation for each kernel variant. This overhead makes benchmark fusion like the compiler evaluates multiple GEMM candidates with different epilogue fusions during scheduling impractical. CudaDSL compiles at speeds comparable to our other backends, removing this constraint and enabling new autotuning strategies.

NVIDIA Optimized GEMM templates. A dedicated team at NVIDIA is actively developing CudaDSL, providing optimized kernel templates for GEMMs and epilogue fusion, and working toward performance parity with the CUTLASS C++ backend. For future generations of hardware, CudaDSL will have an early advantage for hardware-specific optimizations with access to the newest hardware sooner.

In short: Triton handles pointwise well, so our focus for the CudaDSL backend is where the most performance is left on the table like GEMMs, attention, and epilogue fusions on the latest hardware.

Background: How TorchInductor Generates GEMMs

GPU architectures have become extremely complex with the rise of deep learning and AI use cases. As a result of this complexity, there are a lot of choices to make when designing a GEMM kernel such as: tile sizes, warp-specialization, instruction shapes and whether to use asynchronous memory transfers (TMA on Hopper and Blackwell). Torch.compile is uniquely positioned to tackle this problem at runtime because as a JIT compiler, it is able to identify the problem shapes of a model and select the best performing configuration using this information. This technique of tuning a kernel to a specific workload automatically is called autotuning. The flow for the Triton autotuning system for TorchInductor is shown below.

TorchInductor's GEMM autotuning pipeline operates in several stages. When the compiler encounters a matrix multiplication during lowering, it first queries each enabled backend (Triton, CUTLASS, cuBLAS) to determine whether the backend supports the given problem shape, layout, and datatype. Backends that cannot handle the configuration are filtered out at this stage.

For each eligible backend, TorchInductor generates a set of candidate kernels from the backend's template library. These candidates vary in tile size, warp configuration, and other backend-specific parameters. All candidates are then benchmarked on the target hardware, and the fastest kernel is selected.

The selected kernel and its compiled output are written to TorchInductor's cache, so subsequent compilations with the same problem configuration can skip benchmarking entirely. This caching operates at both the individual kernel level (compiled code) and the selection level (which candidate won for a given problem size and backend set).

On top of this base pipeline, TorchInductor supports epilogue fusion for GEMM kernels. During scheduling, the compiler evaluates whether fusing downstream pointwise operations into the GEMM epilogue is profitable. For Triton, this is implemented via the MultiTemplate buffer: the top N GEMM candidates from lowering are carried forward, and possible fusions are benchmarked during scheduling to determine whether a fused variant outperforms the unfused GEMM followed by a separate pointwise kernel. Final kernel selection is deferred until after fusion passes complete. This full flow is shown below.

The CUTLASS C++ backend supports epilogue fusion through the Epilogue Visitor Tree (EVT), but the nvcc compile cost per variant limits the number of configurations that can be practically evaluated. This compile time constraint is one of the primary motivations for introducing CudaDSL as an alternative. Note: epilogue fusion is not supported today in the CudaDSL backend, but this work is planned (see Future Work).

Architecture of the CudaDSL Backend

The CudaDSL backend plugs into the autotuning pipeline described above. When Inductor encounters a matrix

multiplication during lowering, the backend proceeds in three steps: (1) query `cutlass_api` for all kernel configurations compatible with the problem, (2) rank them using `nvMatmulHeuristics` to select the top candidates, and (3) compile and benchmark those candidates on the target hardware alongside Aten and Triton. It differs from the Triton and CUTLASS C++ paths in two key ways.

Kernel selection via `cutlass_api`. The Triton backend generates kernel candidates from templates maintained inside TorchInductor. The CudaDSL backend takes a different approach: it queries `cutlass_api`, an NVIDIA-maintained Python library that contains the full space of CuTeDSL GEMM kernel configurations â€” 200x224 tile shapes, cluster sizes, and scheduling parameters. Inductor describes the problem (shape, dtype, layout, scaling mode, and GPU compute capability) and the API returns all compatible kernels. When NVIDIA adds new kernel configurations or hardware support, they land in `cutlass_api` without changes to Inductor. The API is also extensible: TorchInductor can register its own kernel classes into the same library. We used this to add FP4 GEMM support (NVFP4, MXF4) before it was available upstream â€” 200x224 our vendored kernels go through the same filter, rank, and profile pipeline as NVIDIAâ€™s 200x231s.

Heuristic-guided search space reduction. Querying `cutlass_api` for a given problem can return hundreds of compatible kernel configurations. Benchmarking all of them would be prohibitively expensive. To address this, the CudaDSL backend integrates `nvMatmulHeuristics`, an NVIDIA analytical performance model that scores each configuration by estimated hardware throughput â€” 200x224 accounting for tile efficiency, memory bandwidth, and occupancy. This narrows hundreds of candidates to a handful (5 by default, configurable via `nvgemm_max_profiling_configs`). Only those top-ranked configurations are compiled and benchmarked on the target hardware. Neither Triton nor CUTLASS use an analytical model in this way; they rely on benchmarking over a smaller, template-defined search space.

Once autotuning selects a winning kernel, the compiled artifact is cached in memory â€” 200x224 subsequent calls invoke the compiled function directly with no repeated compilation overhead.

Importantly, the CudaDSL backend is purely additive. If a problem is not compatible with NVGEMM â€” 200x224 unsupported dtype, layout, or hardware â€” 200x224 no NVGEMM candidates are generated, and the autotune proceeds with Aten and Triton as usual. If NVGEMM candidates are generated but lose the benchmark, the faster backend is selected automatically. Enabling NVGEMM cannot cause a performance regression.

Results

All benchmarks were run on a single NVIDIA B200 GPU at 850W with dynamic clocking (no tensor parallelism), using PyTorch nightly and Cuda 13.1.

Kernel-level results measure isolated GEMM latency via Inductor autotuning.

End-to-end results measure vLLM V1 decode latency on Llama 3.1 8B, Qwen3 32B, and Llama 3.3 70B with a 32-token input prompt and 128-token generation, serial execution, and a clean cache between runs.

Kernel-Level Speedups

We evaluated Inductor NVGEMM against existing Inductor backends across three dtype regimes on LLM-relevant GEMM shapes. The charts below show kernel throughput in TFLOPS; callouts indicate speedup over the faster existing backend.

BF16: NVGEMM improves decode-regime shapes ($M = 8$ to $M = 64$, up to 1.73x) and tall-skinny shapes like (4096, 256, 4096) at 1.54x. Prefill-sized shapes are at parity.

MXFP8: NVGEMM improves on medium shapes (up to 1.78x) and is at parity on large shapes. Wide-N rectangular shapes favor Aten.

NVFP4: NVGEMM improves throughput on decode-sized shapes ($M \approx 211 \times 256$), with speedups up to 1.6x over the faster existing backend. At larger M ($\approx 211 \times 512$), Aten is well-tuned and the backends converge.

End-to-End vLLM Inference

We measured inference latency on three models across batch sizes 2â€”200x223128 using vLLMâ€™s 200x231s V1 model runner. Because

vLLM uses dynamic shapes for the batch dimension, Inductor does not know the actual batch size at compile time. We use an `autotune_batch_hint` to specify the target batch size so that Inductor benchmarks kernel candidates at the shape that will be used at runtime â€” 200x224 this is important because optimal kernel configurations are shape-dependent.

BF16: Adding NVGEMM reduces latency on 90% of configurations (19/21 data points). The largest improvement is 6.5% on Llama 3.3 70B at batch size 16. Llama 3.1 8B sees consistent 2â€”200x2234% gains across batch sizes. Qwen3 32B shows more modest improvements of 0.5â€”200x2232.4%.

NVFP4: 89% win rate (16/18 data points). Llama 3.1 8B improves by up to 4.2%, Qwen3 32B by up to 3.5%, and Llama 3.3 70B by up to 3.3%. Gains are most consistent at batch sizes 16â€”200x22364.

CudaDSL Backend Supported Features

How You Can Try It

Installation

The CudaDSL backend requires the `cutlass_api` library, which is currently installed from a specific branch of the CUTLASS repository. `cutlass_api` is expected to merge into the main CUTLASS branch in a future release, at which point this separate installation step will no longer be necessary.

```
bash
# Install CuTeDSL and the matmul heuristics library
pip install nvidia-cutlass-dsl==4.3.5
pip install nvidia-matmul-heuristics

# Clone and install cutlass_api from the cutlass_api branch
git clone --branch cutlass_api https://github.com/NVIDIA/cutlass.git
cd cutlass/python/cutlass_api
pip install -e ".[torch]"
```

You will also need:

â200\223 PyTorch 2.11+ (core NVGEMM support for mm, bmm, scaled_mm, grouped_mm). FP4 kernel support (NVFP4, MXF4) and various performance optimizations requires PyTorch nightly.

Note: cutlass_api currently requires CuTeDSL version 4.3.5 or earlier.

Usage

Once installed, enable the backend by adding NVGEMM to the list of Inductor autotuning backends. Here is a minimal runnable example:

```
import torch
import torch._inductor.config as config

config.max_autotune_gemm_backends = "ATEN,TRITON,NVGEMM"

A = torch.randn(128, 4096, device="cuda", dtype=torch.bfloat16)
B = torch.randn(4096, 4096, device="cuda", dtype=torch.bfloat16)

@torch.compile(mode="max-autotune-no-cudagraphs")
def f(a, b):
    return a @ b

out = f(A, B) # first call triggers autotuning
```

When Inductor encounters a GEMM during compilation, it will evaluate NVGEMM kernel candidates alongside Aten and Triton and select the fastest. Operations that NVGEMM does not support automatically fall back to other backends.

The same configuration can be set via environment variable:

```
TORCHINDUCTOR_MAX_AUTOTUNE_GEMM_BACKENDS="ATEN,TRITON,NVGEMM" python my_script.py
```

To control how many kernel configurations are profiled per GEMM:

```
config.nvgemm_max_profiling_configs = 10 # default is 5; set to None for all
```

Future Work

The following items represent the planned development roadmap for the CuteDSL backend.

Benchmark epilogue fusion. With CuteDSL compile times no longer a bottleneck, TorchInductor can benchmark epilogue fusion decisions for GEMM kernels. This is important because replacing cublas for an individual GEMM may not always be profitable, so epilogue fusion provides an avenue to consistently outperform cublas which canâ200\231t perform any fusions. This work involves deferring final kernel selection until after fusion passes complete, evaluating fused and unfused variants across backends, and selecting the globally optimal configuration. cutlass_api already provides epilogue-fusion-capable (EFC) kernels supporting auxiliary tensor loads/stores, elementwise operations (addition, multiplication, subtraction, division), and activations (relu, sigmoid, tanh). The remaining work is on the Inductor side: mapping Inductorâ200\231s fusion decisions to the EFC kernel interface and integrating them into the scheduling pipeline. Additional epilogue operations â200\224 reductions and row/column broadcasts â200\224 are planned in cutlass_api.

Async Parallel precompilation and persistent caching. Currently, kernel candidates are compiled sequentially inline via cute.compile(). We are adding parallel precompilation across subprocesses and persistent on-disk caching of compiled artifacts, so that warm autotuning runs can skip compilation entirely.

Exportable configuration caches. A portable, human-readable format (JSON or protobuf) for autotuned GEMM configurations, with import/export APIs for cache manipulation. This enables configuration portability across autotuning runs and environments.

FlexAttention-style matmul API. A higher-order API allowing users to specify backend preferences, tile configurations, and epilogues at the matmul callsite. This would provide explicit control over autotuning behavior and interoperate with the exportable configuration cache.

Quack GEMM Integration. Tri Daoâ200\231s Quack library has optimizer blackwell GEMM implementations, we will investigate to see how the performance compares to our current templates, and integrate these templates if they are more performant.

AOT compilation support. For inference deployments, precompiling CuteDSL kernels at export time would eliminate autotuning overhead. This depends on a precompile API planned for the CuteDSL 4.4 release and will require investigation into C++ accessibility for AOTI integration.

CUTLASS C++ backend replacement. On newer hardware generations, CuteDSL is expected to reach full parity with the C++ backend. At that point, CuteDSL would serve as a replacement, simplifying the TorchInductor codebase by consolidating the CUTLASS integration into a single Python-based path.

Conclusion

In this post, we presented the architecture of the TorchInductor CuteDSL backend, how to enable it today, and our benchmarking results. As shown in the future work, this is the first presentation of this work and there is a lot more to do. If there are any issues, questions, or new ideas youâ200\231d like to see with the new backend, file an issue on github and tag us!

Docs

Access comprehensive developer documentation for PyTorch

View Docs â200°

Tutorials

Get in-depth tutorials for beginners and advanced developers

[View Tutorials](#)

Resources

Find development resources and get your questions answered

[View Resources](#)

Stay in touch for updates, event info, and the latest news

By submitting this form, I consent to receive marketing emails from the LF and its projects regarding their events, training, research, developments, and related announcements. I understand that I can unsubscribe at any time using the links in the footers of the emails I receive. [Privacy Policy](#).

- * [x-twitter](#)
- * [facebook](#)
- * [linkedin](#)
- * [youtube](#)
- * [github](#)
- * [slack](#)
- * [discord](#)

© 2026 PyTorch. Copyright © The Linux Foundation. All rights reserved. The Linux Foundation has registered trademarks and uses trademarks. For more information, including terms of use, privacy policy, and trademark usage, please see our [Policies page](#). [Trademark Usage](#). [Privacy Policy](#)

Close Menu

- * Learn
 - + [Get Started](#)
 - + [Tutorials](#)
 - + [Learn the Basics](#)
 - + [PyTorch Recipes](#)
 - + [Intro to PyTorch](#) [2023 YouTube Series](#)
 - + [Videos](#)
 - + [PyTorch Certification](#)
- * Community
 - + [Landscape](#)
 - + [Join the Ecosystem](#)
 - + [Community Hub](#)
 - + [Forums](#)
 - + [Developer Resources](#)
 - + [Events](#)
 - + [Working Groups](#)
 - + [Meeting Calendar](#)
 - + [Contributor Awards](#)
 - + [Ambassadors](#)
- * Projects
 - + [PyTorch](#)
 - o [Executorch](#)
 - + [vLLM](#)
 - + [DeepSpeed](#)
 - + [Ray](#)
 - + [Helion](#)
 - + [Safetensors](#)
 - + [Host Your Project](#)
- * Docs
 - + [PyTorch](#)
 - + [Domains](#)
- * Blog & News
 - + [Blog](#)
 - + [Announcements](#)
 - + [Case Studies](#)
 - + [Newsletter](#)
- * About
 - + [PyTorch Foundation](#)
 - + [Members](#)
 - + [Governing Board](#)
 - + [Technical Advisory Council](#)
 - + [Cloud Credit Program](#)
 - + [Staff](#)
 - + [Contact](#)
 - + [Brand Guidelines](#)
- * JOIN